Postgres Pool Metrics Runner

This workflow lets you compute pool metrics in Postgres (fast window functions) without changing the 520 pipeline.
You can push a pandas DataFrame to Postgres, run the SQL, then pull the results back into pandas.

If you want CELL 6D to load Postgres results directly, enable: `USE_POSTGRES_POOL_METRICS=True` in `pipeline_config.py` and set the connection/table variables there.

Files

- `postgres_pool_metrics.sql`: SQL that computes rater + senior-rater pool metrics
- `run_pool_metrics_postgres.py`: Python wrapper that executes the SQL via SQLAlchemy

1) Push the pre-pool DataFrame to Postgres

Use the pre-pool checkpoint from Cell 6C (`df_pipeline_03c_pre_pool_metrics`) and write it to Postgres.
Optional: set `POSTGRES_POOL_PUSH_6C=True` and `POSTGRES_POOL_INPUT_TABLE` in `pipeline_config.py` to auto-push during CELL 6C.

```
import pandas as pd
from sqlalchemy import create_engine

PG_URL = "postgresql+psycopg2://USER:PASSWORD@HOST:5432/DBNAME"
engine = create_engine(PG_URL)

# df_pre_pool = load_feather('df_pipeline_03c_pre_pool_metrics')
df_pre_pool.to_sql(
    "df_pipeline_03c_pre_pool_metrics",
    engine,
    schema="public",
    if_exists="replace",
    index=False
)
```

2) Run the SQL in Postgres

Option A: run via Python (choose modes):

```
python run_pool_metrics_postgres.py
```

Option B: copy/paste `postgres_pool_metrics.sql` into pgAdmin and execute.
Replace `{source_table}` and `{target_table}` at the top of the SQL before running.

3) Pull the results back into pandas

```
import pandas as pd
from sqlalchemy import create_engine

PG_URL = "postgresql+psycopg2://USER:PASSWORD@HOST:5432/DBNAME"
engine = create_engine(PG_URL)

df_pool = pd.read_sql(
    "SELECT * FROM public.df_pipeline_03_oer_metrics_pg",
    engine
)
```

Notes

- Pool membership is based on `snpsht_dt` + `rater` / `snr_rater` in the pre-pool table.
- Rank percentiles use **average rank** within ties, consistent with pandas `rank(pct=True, method='average')`.
- Z-scores use `STDDEV_POP` (ddof=0), matching NumPy defaults.
- If you want to include/exclude self in pool means, set `EXCLUDE_SELF` in `run_pool_metrics_postgres.py`.
- Default pool min size is controlled by `MIN_POOL_SIZE` in `run_pool_metrics_postgres.py`.
- **Mode switches**:
  - `SELF_MODE`: which columns to use for the individual values in diffs (`base`, `fwd`, `bwd`)
  - `POOL_MODE`: which columns to use to build pool distributions (`base`, `fwd`, `bwd`)
  - Mixed modes are allowed (e.g., `SELF_MODE="fwd"` and `POOL_MODE="bwd"`).

Postgres workflow checklist (520 + config + SQL)

1) Files in place - `postgres_pool_metrics.sql` and `run_pool_metrics_postgres.py` in the working directory - `520_pipeline_cox_working.ipynb` and `pipeline_config.py` are the active versions

2) `pipeline_config.py` settings - `POSTGRES_POOL_PG_URL` → valid SQLAlchemy URL - `POSTGRES_POOL_INPUT_TABLE` → where CELL 6C pushes - `POSTGRES_POOL_OUTPUT_TABLE` → where SQL writes pool metrics - `POSTGRES_POOL_PUSH_6C = True` if you want auto-push in CELL 6C - `POSTGRES_POOL_RUN_SQL = True` if you want CELL 6C to run the SQL - `USE_POSTGRES_POOL_METRICS = True` if you want CELL 6D to load from Postgres

3) Run 520 up to CELL 6C - CELL 6C creates `df_pipeline_03c_pre_pool_metrics` - If `POSTGRES_POOL_PUSH_6C=True`, it will push to `POSTGRES_POOL_INPUT_TABLE`

4) Run SQL to compute pool metrics - Option A: `python run_pool_metrics_postgres.py` - Option B: paste `postgres_pool_metrics.sql` into pgAdmin - Confirm output table = `POSTGRES_POOL_OUTPUT_TABLE` - Optional: enable `POSTGRES_POOL_RUN_SQL=True` to run this from CELL 6C

5) Run CELL 6D - If `USE_POSTGRES_POOL_METRICS=True`, CELL 6D loads from Postgres - Otherwise, it will compute in pandas (slow path)